# CSC 710 Software Engineering

Instructor: Dr. Tianxiao Zhang

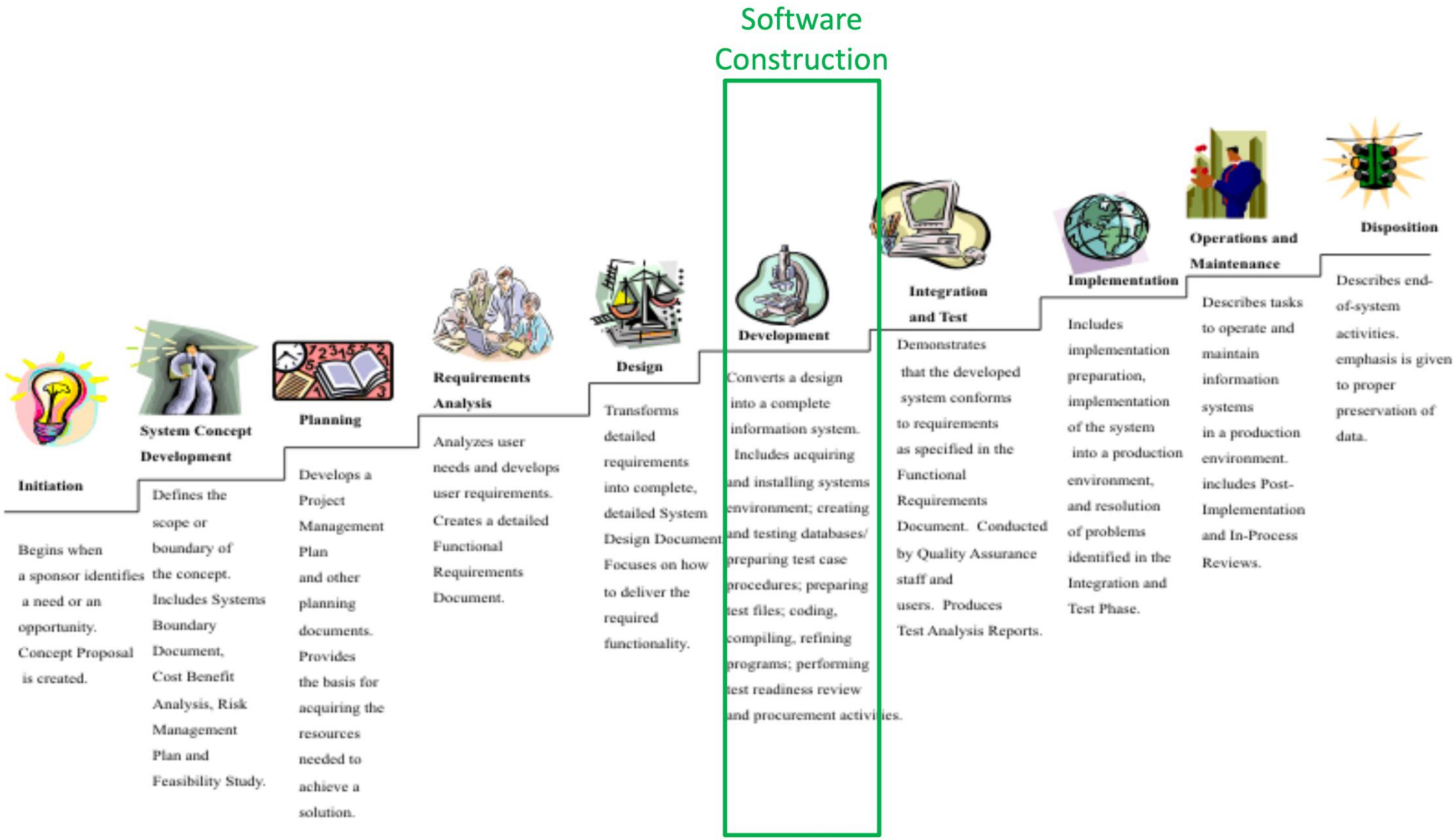
# Software Construction

- SWEBOK V3.0 (2014) is the most recent version of the IEEE Guide to the Software Engineering Body of Knowledge.
- According to SWEBOK Version 3:
  - Software construction is a software engineering discipline.
  - It is the detailed creation of working meaningful software through a combination of coding, verification, unit testing, integration testing, and debugging.
  - It is linked to all the other software engineering disciplines, most strongly to software design and software testing.

# Software Construction

- ACM separates these activities differently
  - Software Construction: coding and debugging
  - Software Verification and Validation: verification, unit testing, and integration testing

# Software Development Phases



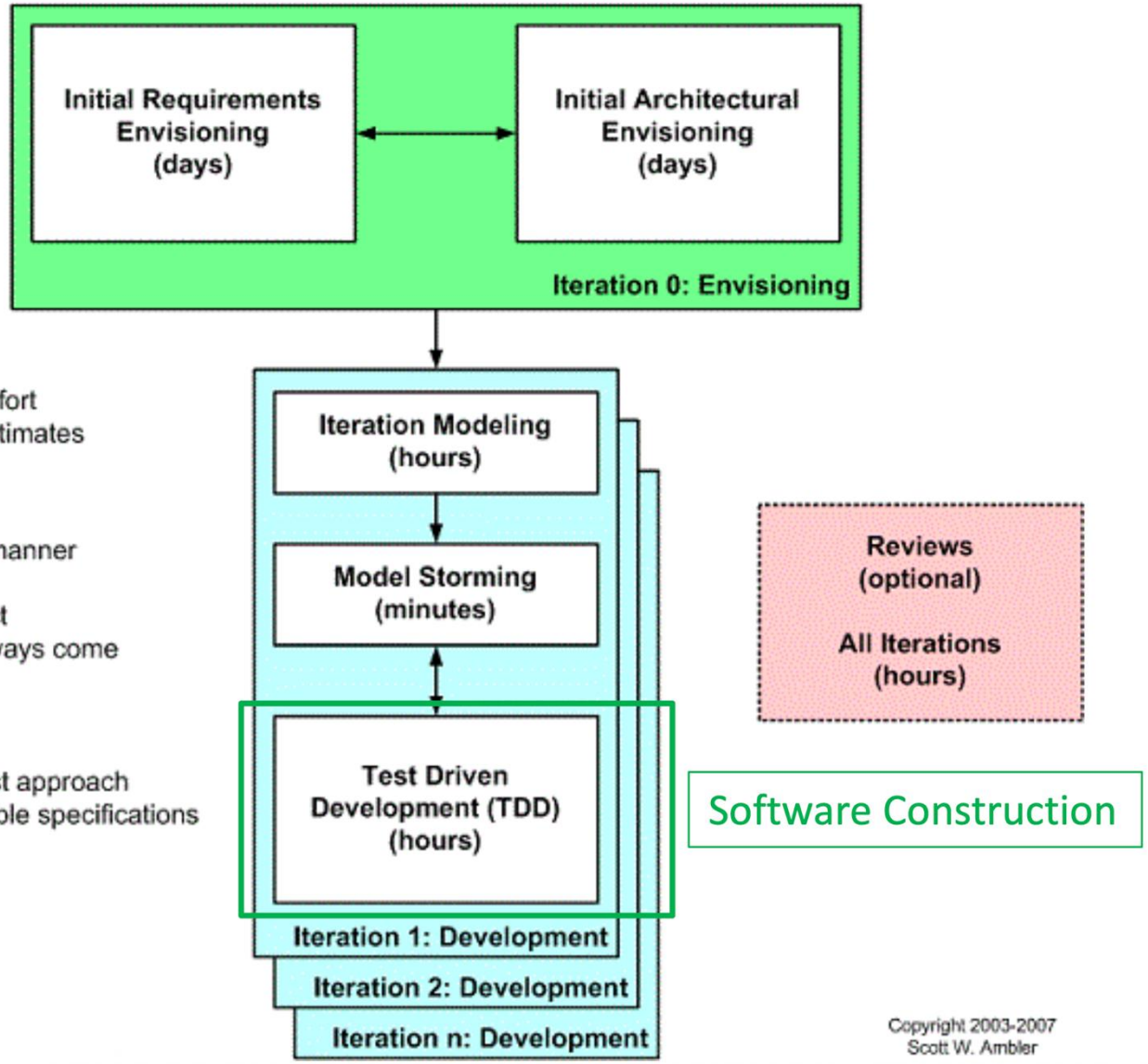
# Agile Model Driven Development (AMDD)

- Identify the high-level scope
- Identify initial "requirements stack"
- Identify an architectural vision

- Modeling is part of iteration planning effort
- Need to model enough to give good estimates
- Need to plan the work for the iteration

- Work through specific issues on a JIT manner
- Stakeholders actively participate
- Requirements evolve throughout project
- Model just enough for now, you can always come back later

- Develop working software via a test-first approach
- Details captured in the form of executable specifications




Copyright 2003-2007
Scott W. Ambler

# Software Construction Topics

- **Greenfield & Brownfield**
- Commenting
- Design by Contract (DbC)

# Types of Projects

- There are two types of software development projects called:
  - Greenfield Development
  - Brownfield Development
- These terms are also used in the construction and financial industry and have similar meanings.

# Greenfield Development

- Greenfield Development happens when you start a brand new project, as in, clean slate development.
- No legacy code lying around, no old development to maintain.
- You’re starting a fresh, from scratch, from a blank slate, with no restrictions on what you’re doing (other than business rules of course).
- Project which lacks any constraints imposed by prior work.
- True Greenfield projects are quite rare.
- Almost always you have to interact with some legacy code at some point in time and perform integrations with that legacy code that impose limitations on your development.

# Greenfield Development

Examples of Greenfield Development

- Most of your programming assignments in college
- CSC 710 Project 1
- Building a new E-commerce solution from the ground up
- Implementing a new rules engine for your company from the ground up.
- Rewriting an old app in a new language (maybe C++ to C#) but not utilizing any of the old code other than for business rule reference.
- A new startup wants application X Built.
  - Application X has never been built before
  - Therefore Application X is Greenfield Development

# Brownfield Development

- Brownfield Development happens when a business decides to improve upon an existing application infrastructure.
- As an upgrade is implemented into an existing solution, the development is said to be Brownfield.
- Working with previously created code.
- Not maintenance programming
- Not fixing bugs
- Just like Greenfield Development, you are adding new functionality
- It is generally more challenging (and fun) because you have to add new functionality within the constraints of the existing system

# Brownfield Development

Examples of Brownfield Development:

- Probably what most Software Engineers do in their jobs
- CSC 710 Project 2
- Adding a new feature to software that was developed last year
- Altering the functionality of the code to enhance the performance of an application
- Upgrading the codebase to expand the working set of functionality across the board.
- Adobe Photoshop and Microsoft Word
- Even modern video games, such as Call of Duty: Black Ops 3 and StarCraft 2, were created from earlier versions of the software.

# Greenfield Development Risks

- While Greenfield projects are open ended, developing software from scratch involves inherent risk.
- For example, there may not be as large of a market for a program as the developer expects.
- The interface may not be well-received and may need to be altered or redesigned to be more user-friendly.
- It may take several updates before a Greenfield application is successful in the marketplace.
- Of course, Greenfield programs that succeed often benefit from being a unique option for users
- … until similar applications are developed by the competitors

# Brownfield Development Advantages

- Brownfield software development has many advantages over Greenfield projects.
- For instance, whenever a software company releases an update to a program, they have a good idea of the market for the software.
- They also know what features and style of interface their users expect.
- Adding new features and interface enhancements is less time consuming than developing a program from scratch.
- Therefore, Brownfield development is less costly and involves less risk than Greenfield development.

# Brownfield Development

- Writing code for a Greenfield project is easier than for a Brownfield project.
- You’ve done this in most of your courses.
- Brownfield development is analyzing and implementing changes to an existing code base, large or small.
- This is not easy.

# Brownfield Development

Here are a few suggestions:

1. Try to find out what the code is supposed to do, in business terms.
2. Read all the documentation that exists, no matter how bad it is.
3. Talk to anyone who might know something about the code.
4. If there are automated tests, look at what the tests are testing.
5. Refactor it (more about this later).
6. Read the comments.
7. Add comments to explain how you think the code works.
8. Step through the code in the debugger, or use print statements
9. Introduce small changes and see what breaks.
10. Make small changes to the code to make it clearer.

# Software Construction Topics

- Greenfield & Brownfield
- **Commenting**
- Design by Contract (DbC)

# Commenting Your Code?

"What a waste of time!"

"What the code does should be intuitively obvious to any good programmer who knows the language."

# Commenting Your Code ? YES!!!

What code does, is not “intuitively” obvious!

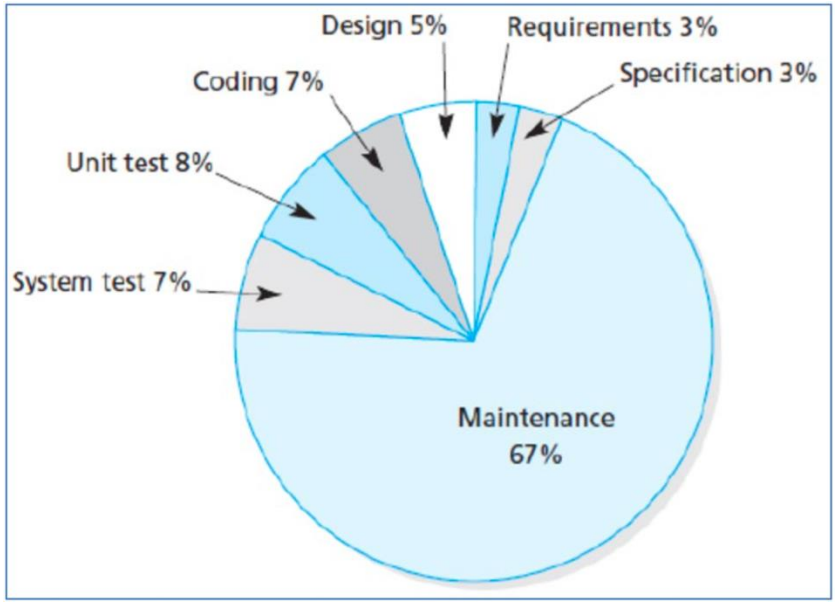
Comments help Brownfield programmers

And remember you will more than likely be a “Brownfield Programmer”

Comments also help <u>you</u> remember why you coded something the way you did.

# Comments – Why?

- Reducing the cost of software maintenance is the most often cited reason for commenting programs.
  - 67% of the lifetime cost of a piece of software goes to maintenance.
- Hardly any software is maintained for its whole life by the original author.
- Commenting improves the readability of the software, allowing engineers to understand new code more quickly and thoroughly.



# Comments

**Prologue Comments**

- Name of program contained in the file
- Brief description of the program
- Inputs
- Output
- All collaborators
- Other sources for the code ChatGPT, stackOverflow, etc.
- Author’s full name
- Creation date
- Revision dates

# Comments

**In-Line Comments**

- Provide comments that explain what each line of code is doing.
- You may comment each line of code and/or provide a multi-line comment that explains what a group of lines does.
- Multi-line comments should be detailed enough that it is clear what each line of code is doing.
- Each block of code must indicate whether you authored the code, you obtained it from one of the sources listed in the prolog, or one of your collaborators authored the code, or if it was a combination of all of these.

# Prologue Comments

- Name of program contained in the file (e.g., CSC 710 Project 1)
- Brief description of the program, e.g.,
  – Battleship game
- Inputs (e.g., none, for a function, it would be the parameters passed to it)
- Output, e.g.,
  – Browser window with 2 test buttons
- All collaborators
- Other sources for the code ChatGPT, stackOverflow, etc.
- Author's full name
- Creation date: The date you first create the file, i.e., the date you write this comment
- Revision dates: when the file was revised and by whom

# In-Line Comments

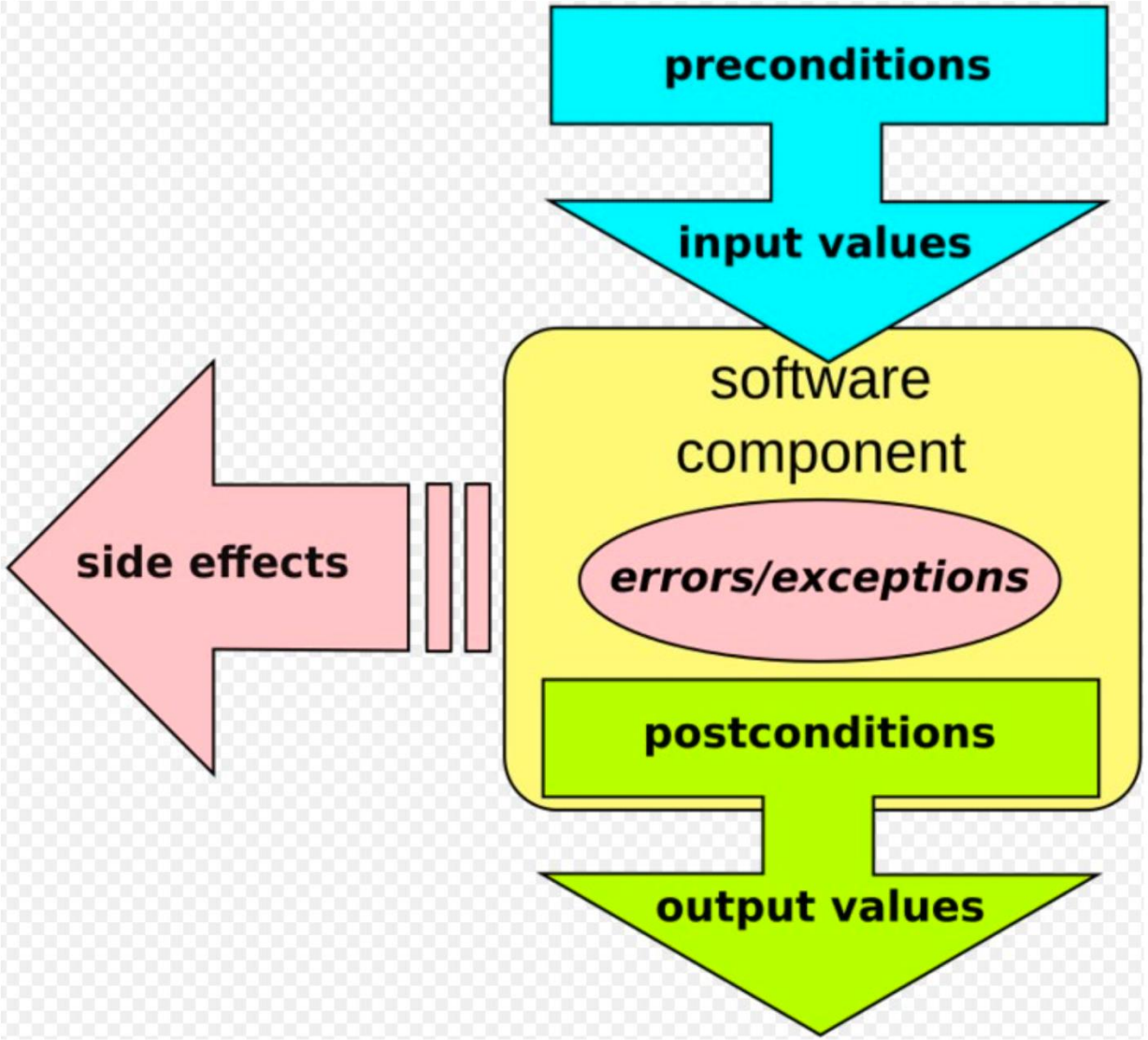
- Provide comments that explain what each line of code is doing.
- You may comment each line of code (e.g., using //) and/or provide a multi-line comment (e.g., using /* and */) that explains what a group of lines does.
- Multi-line comments should be detailed enough that it is clear what each line of code is doing.
- Each block of code must indicate whether you authored the code, you obtained it from one of the sources listed in the prolog, or one of your collaborators authored the code, or if it was a combination of all of these.

# Software Construction Topics

- Greenfield & Brownfield
- Commenting
- **Design by Contract (DbC)**

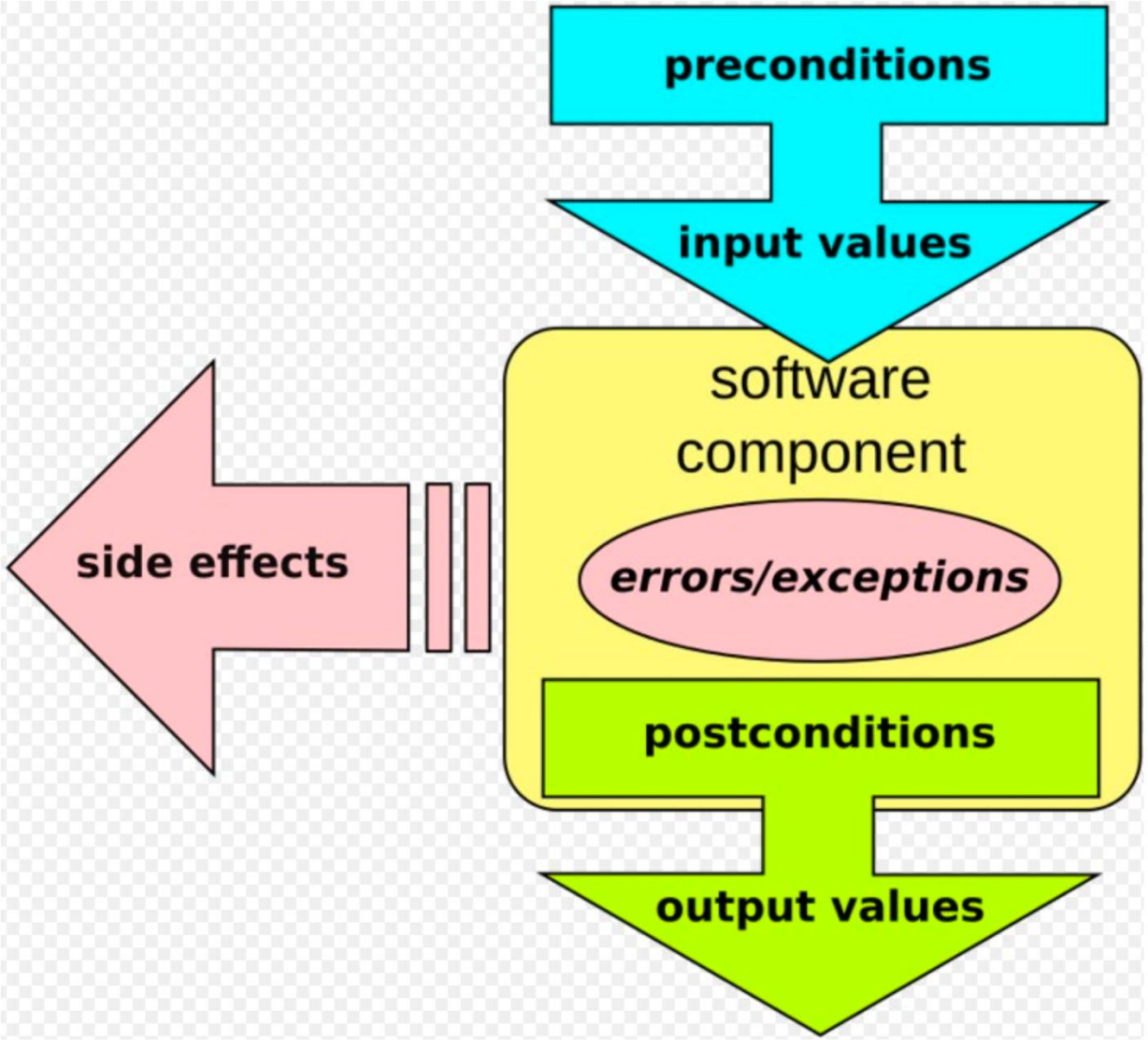
# Design by Contract (DbC)

- Design by contract (DbC), also known as contract programming, programming by contract, and design-by-contract programming, is an approach for designing software.
- Formal, precise and verifiable interface specifications for software components.



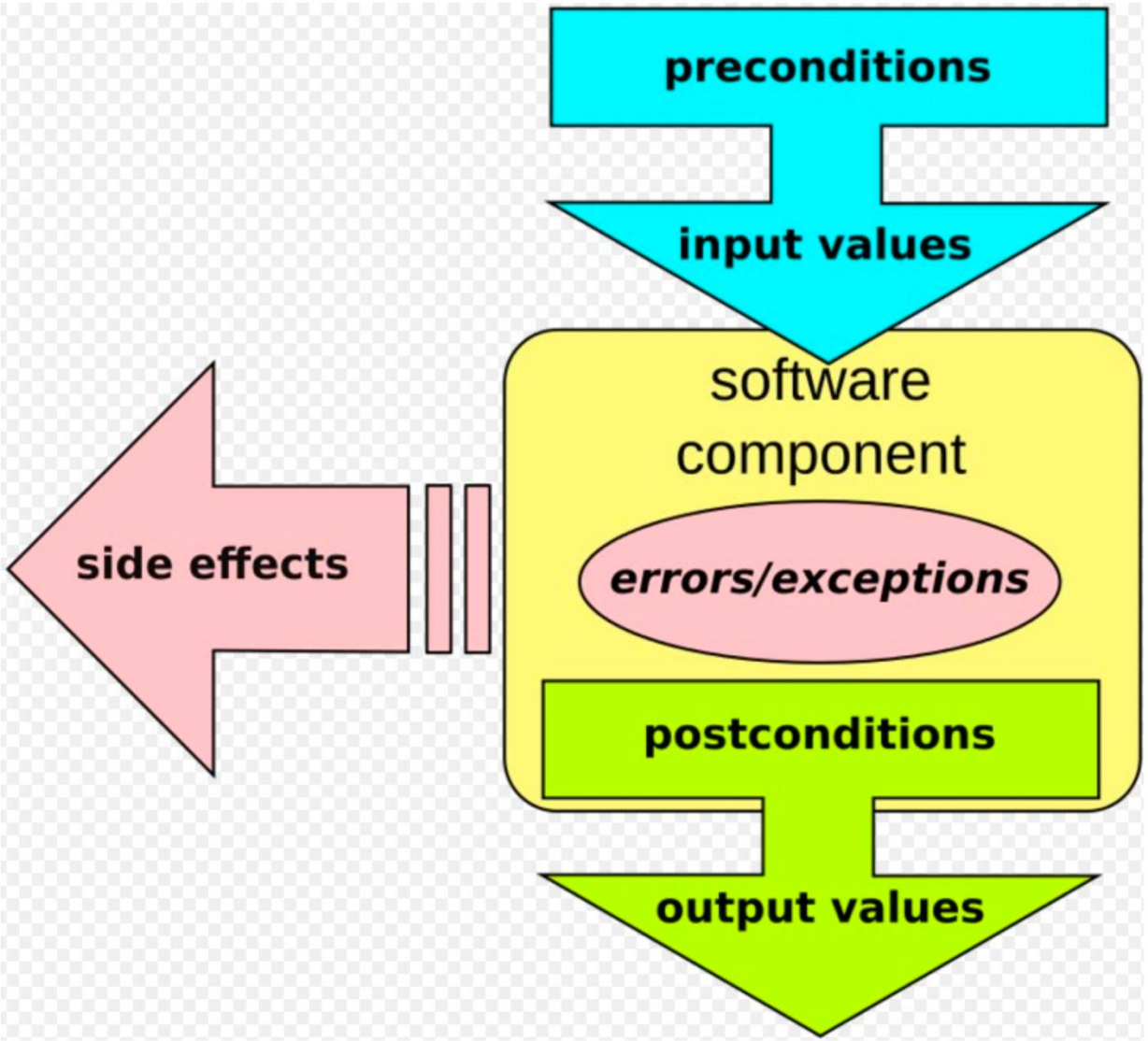
# Design by Contract (DbC)

- Extends the ordinary definition of abstract data types with:
  - Preconditions
  - Postconditions
  - Side effects
  - Invariants
  - Errors/exceptions
- These specifications are referred to as “contracts”.



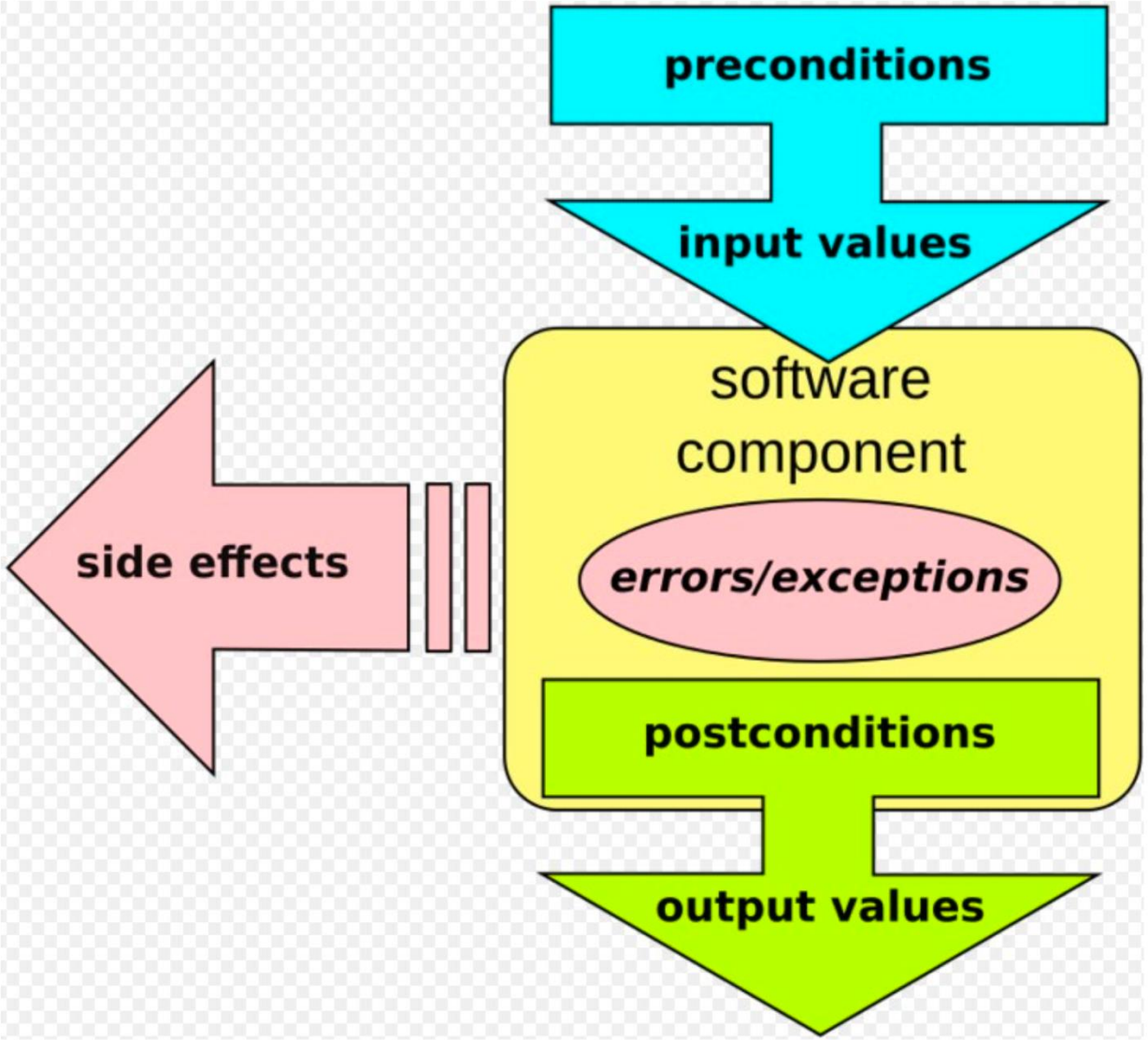
# DbC History

- The term was coined by Bertrand Meyer in connection with his design of the Eiffel programming language.
- Eiffel Software was granted a trademark for DbC December 2004.



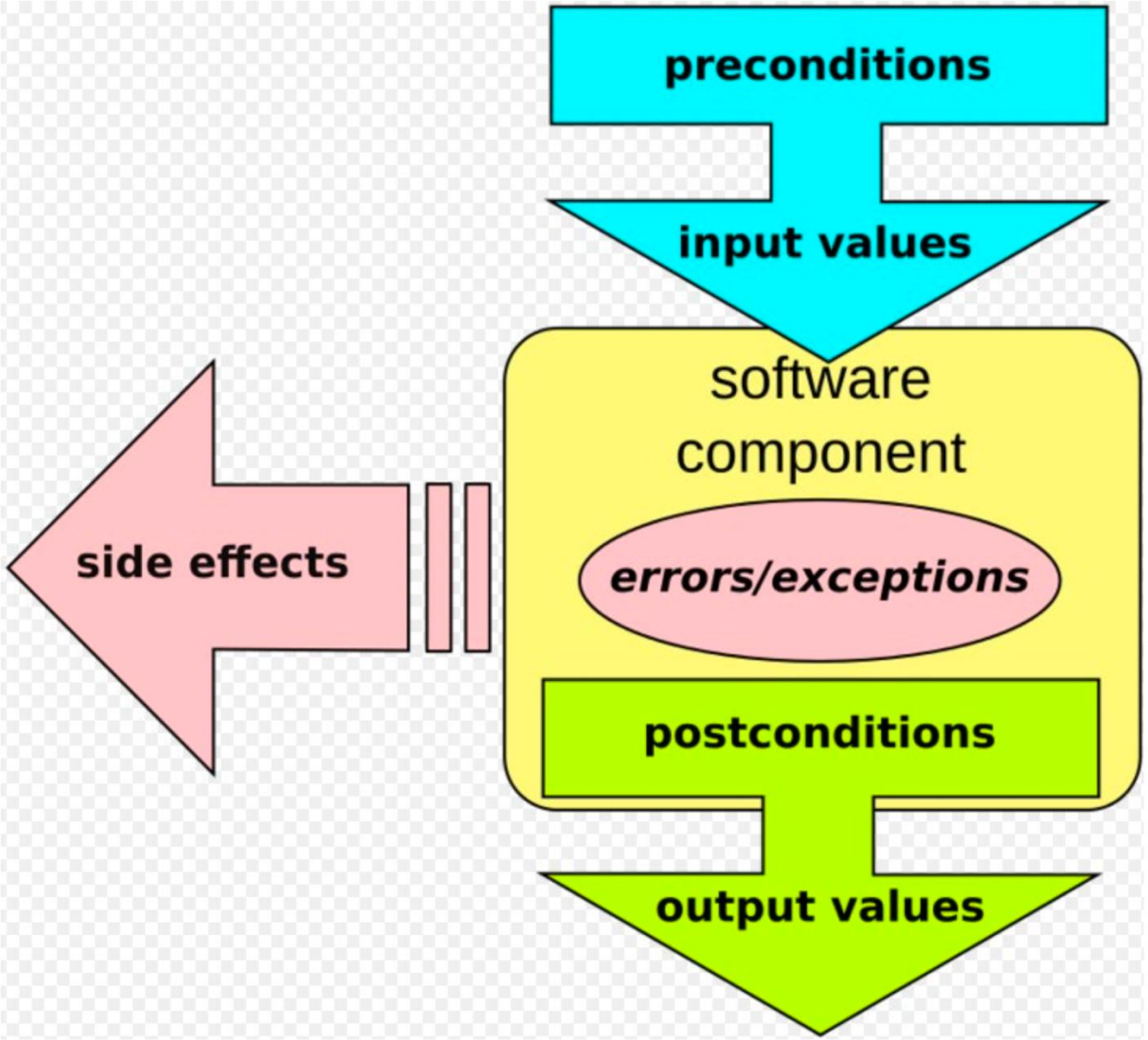
# DbC History

- Eiffel is an object oriented programming language designed by Bertrand Meyer (an object-orientation proponent and author of Object-Oriented Software Construction) and Eiffel Software.



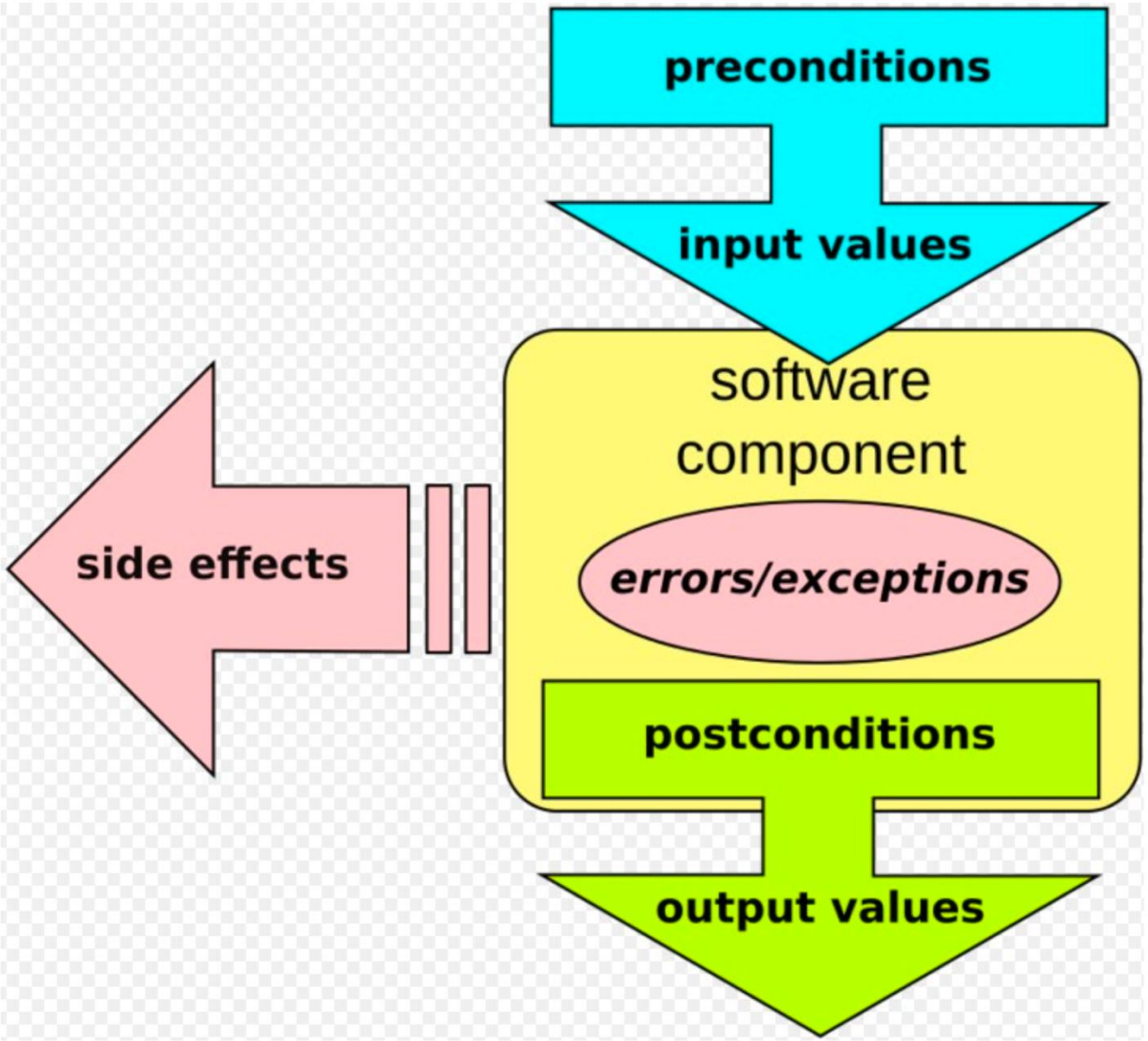
# DbC History

- Meyer conceived the language in 1985 with the goal of increasing the reliability of commercial software development.
- In 2005, Eiffel became an ISO-standardized language.
- The design of the language is closely connected with the Eiffel programming method.



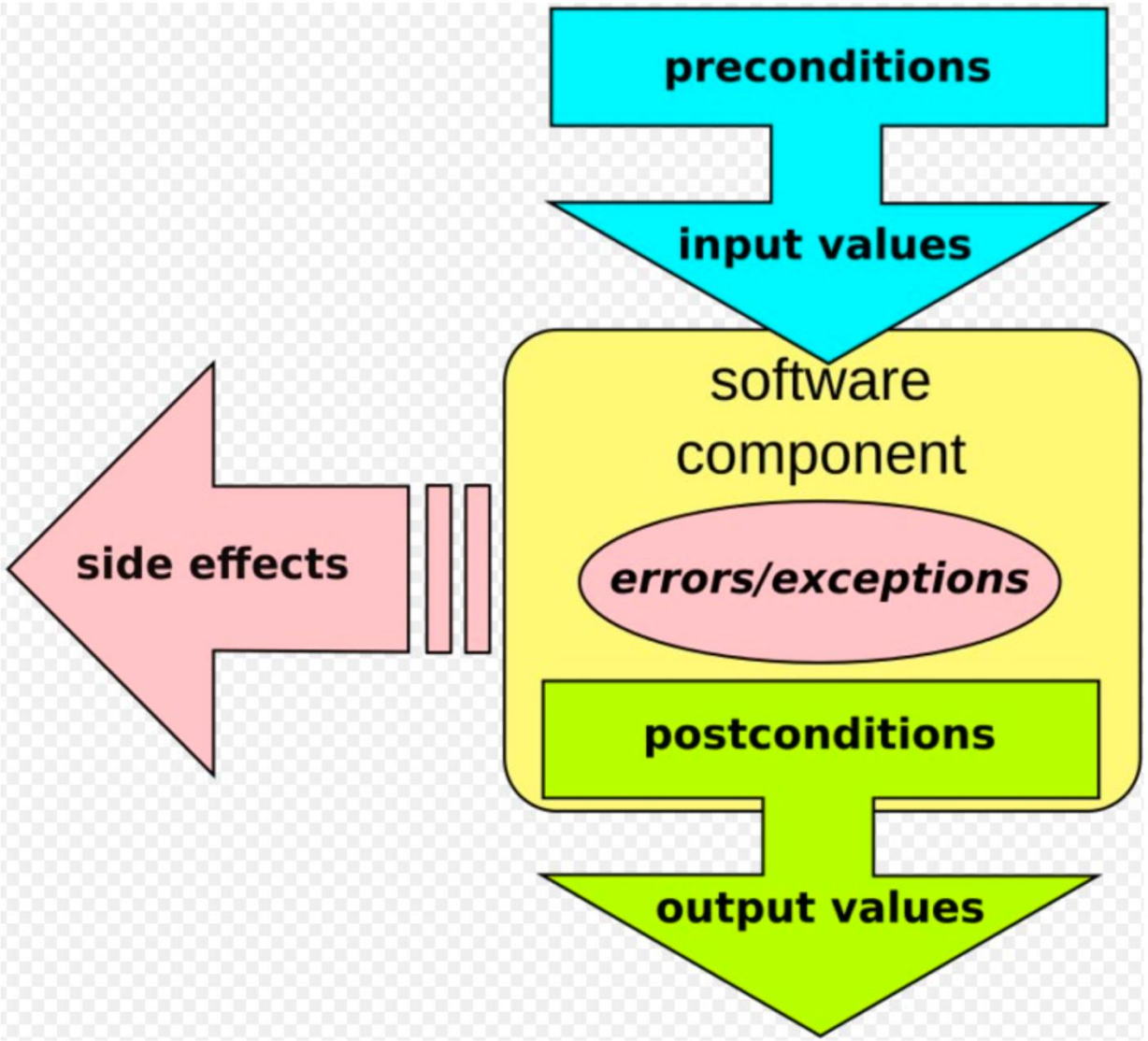
# DbC History

- Both are based on a set of principles, including design by contract, command–query separation, the uniform-access principle, the single-choice principle, the open–closed principle, and option–operand separation.



# DbC History

- Many concepts initially introduced by Eiffel later found their way into Java, C#, and other languages.
- New language design ideas, particularly through the Ecma/ISO standardization process, continue to be incorporated into the Eiffel language.

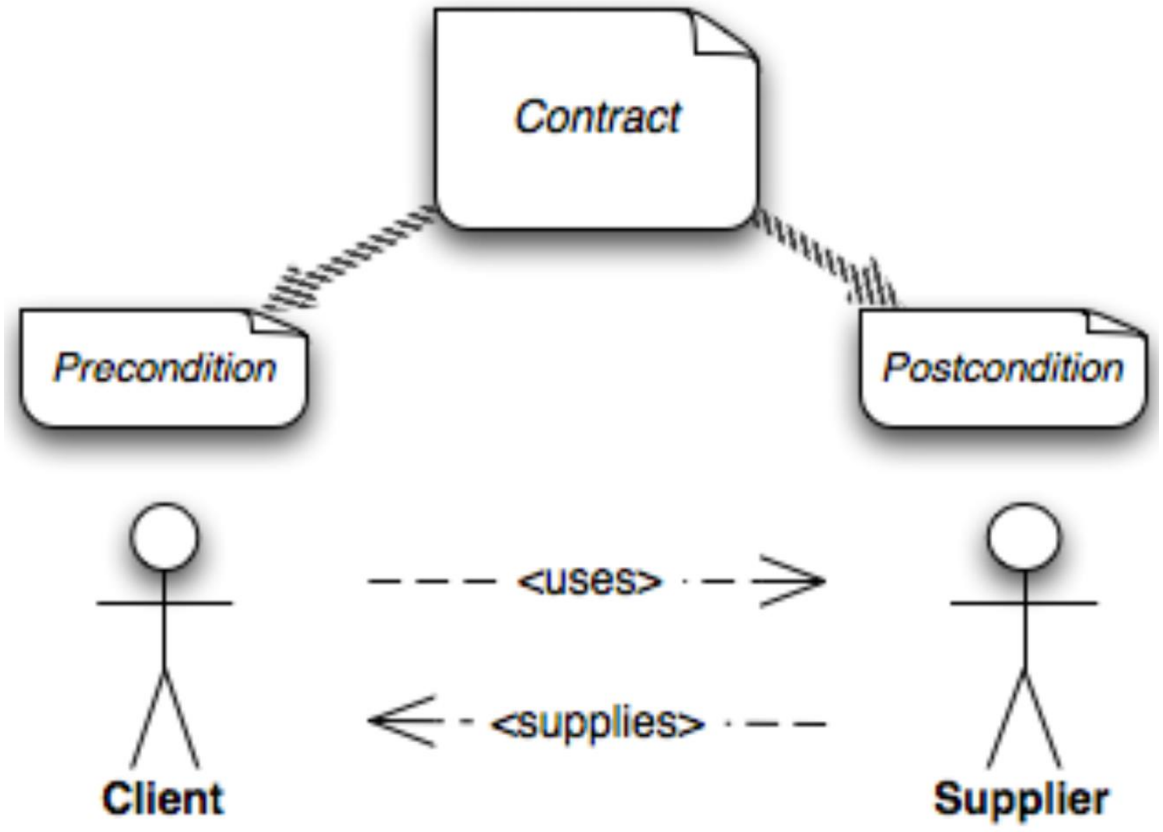


# Design by Contract (DbC) Metaphor

- Contract between client software and supplier software

- Client software agrees to provide correct inputs (e.g., function parameters) in exchange for valid output (e.g., return values).

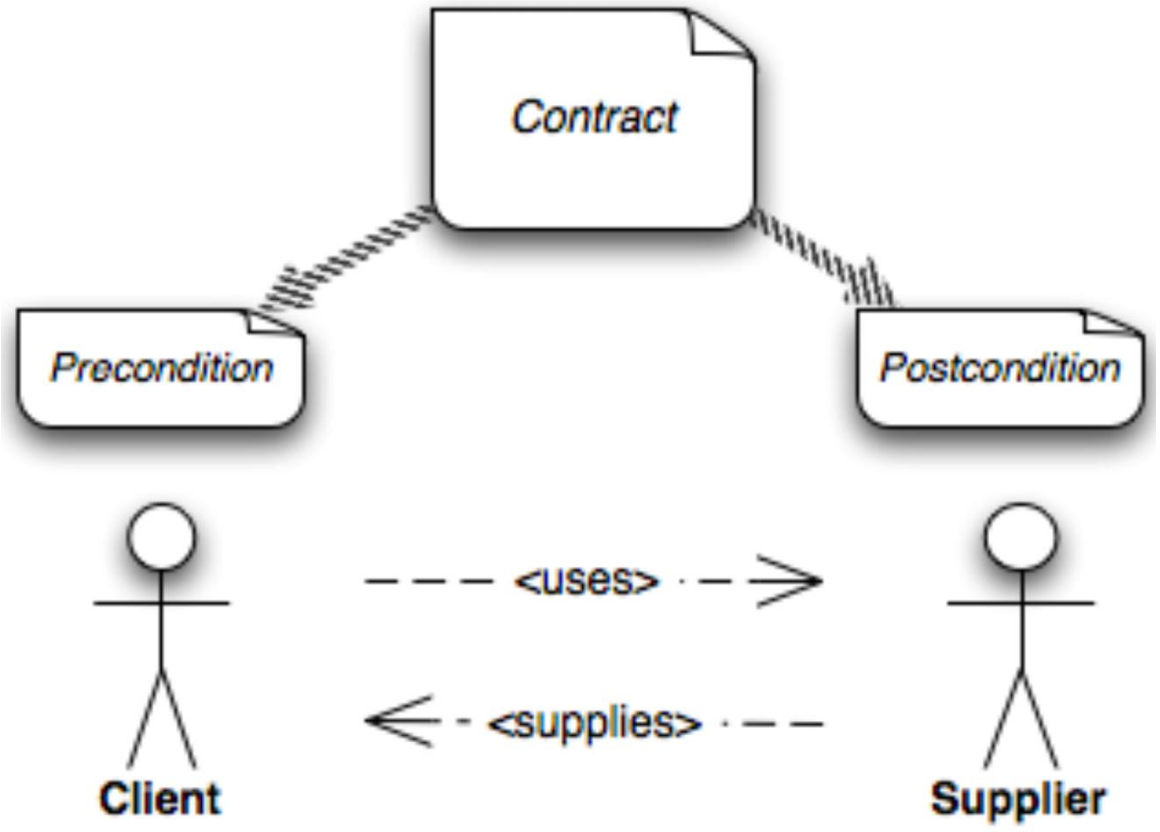
- Supplier software agrees to provide valid output if client software correctly provides inputs.



# What's in a DbC Contract?

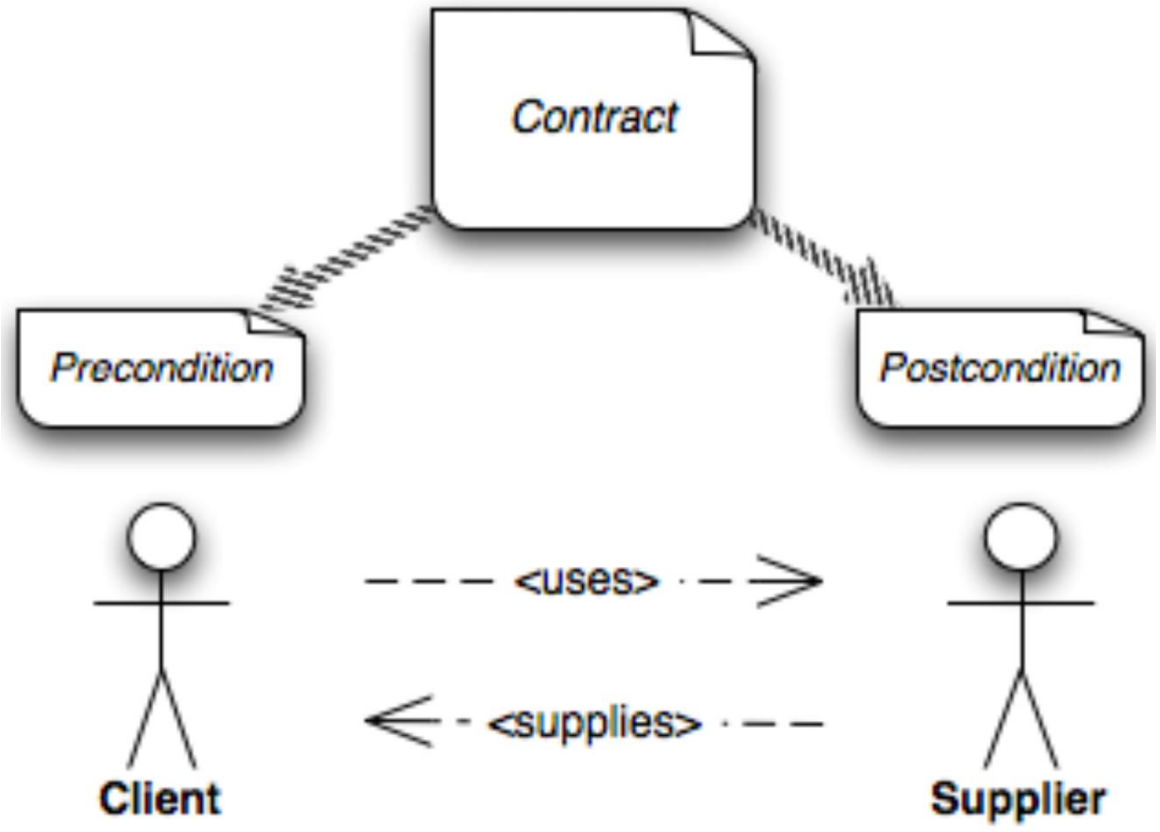
The contract for each component (e.g., method) will normally contain the following pieces of information:

- Preconditions
  – Acceptable and unacceptable input values or types, and their meanings
- Postconditions
  – Return values or types, and their meanings
- Side effects - next slide
- Invariants – slide after next
- Errors/exceptions
  – values or types that can occur, and their meanings



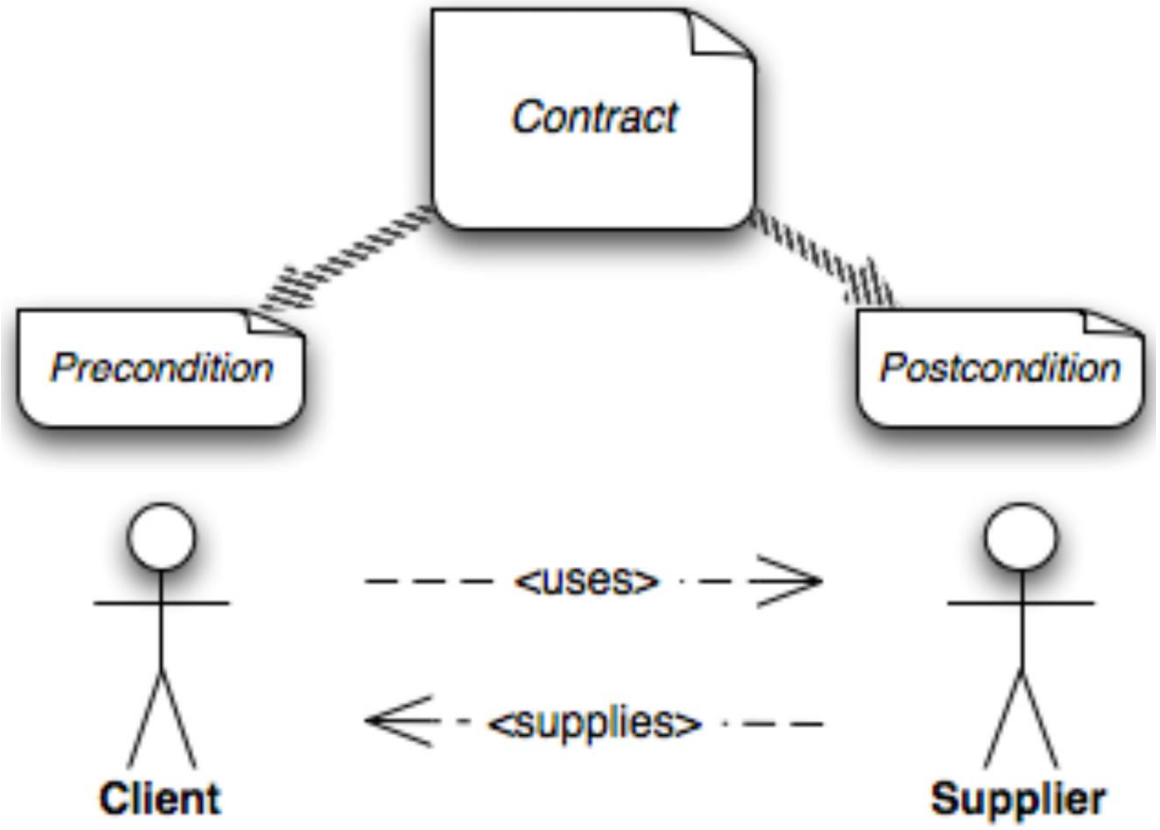
# What's a Side Effect?

- An operation, function or expression is said to have a side effect if it modifies something outside its local environment.
- For example:
  – modifying a global variable
  – modifying a static local variable
  – modifying a mutable argument passed by reference
  – performing I/O
  – calling other side-effect functions



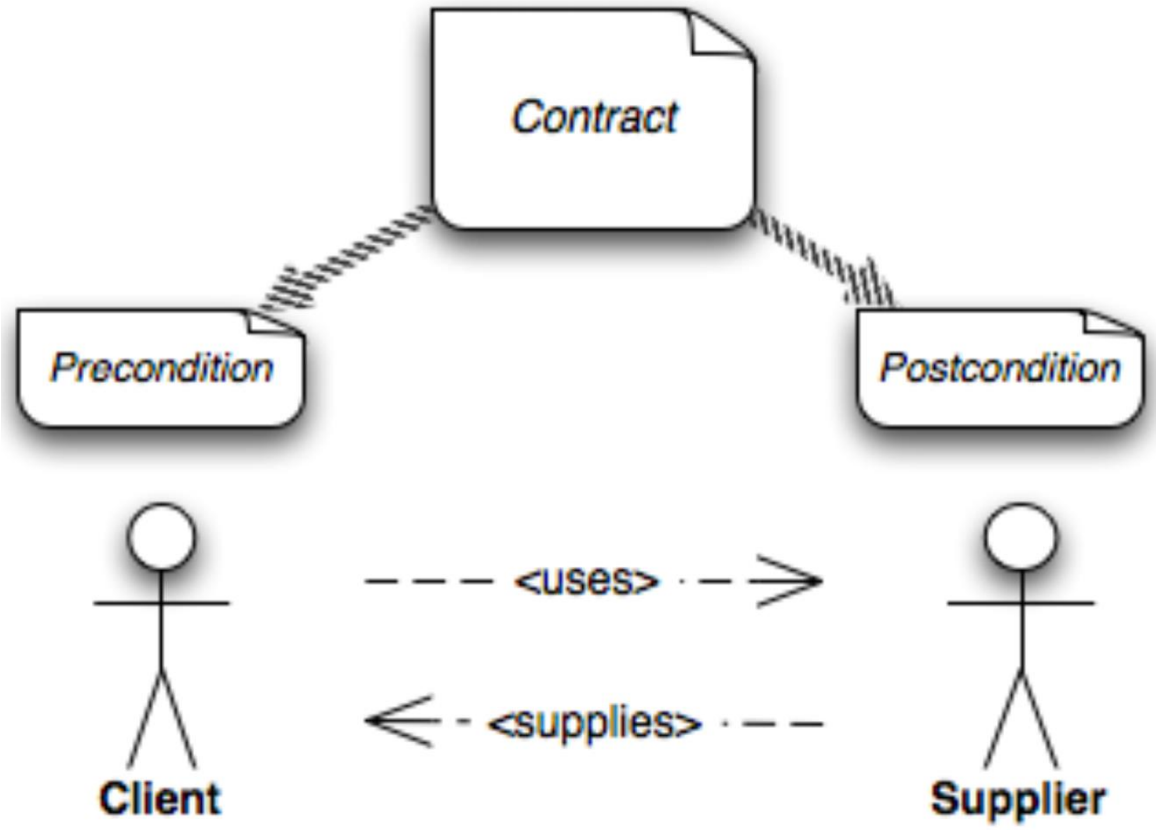
# What's an Invariant?

- Formally from Discrete Structures:
  – An invariant with respect to an equivalence relation is a property that is constant on each equivalence class.
- Informally:
  – An invariant is a condition or relation that is always true.



# What’s an Invariant?

- For example:
  – A binary search tree might have the invariant that for every node, the key of the node's left child is less than the node's own key.
  – A correctly written insertion function for this tree will maintain that invariant.



# Offensive Programming

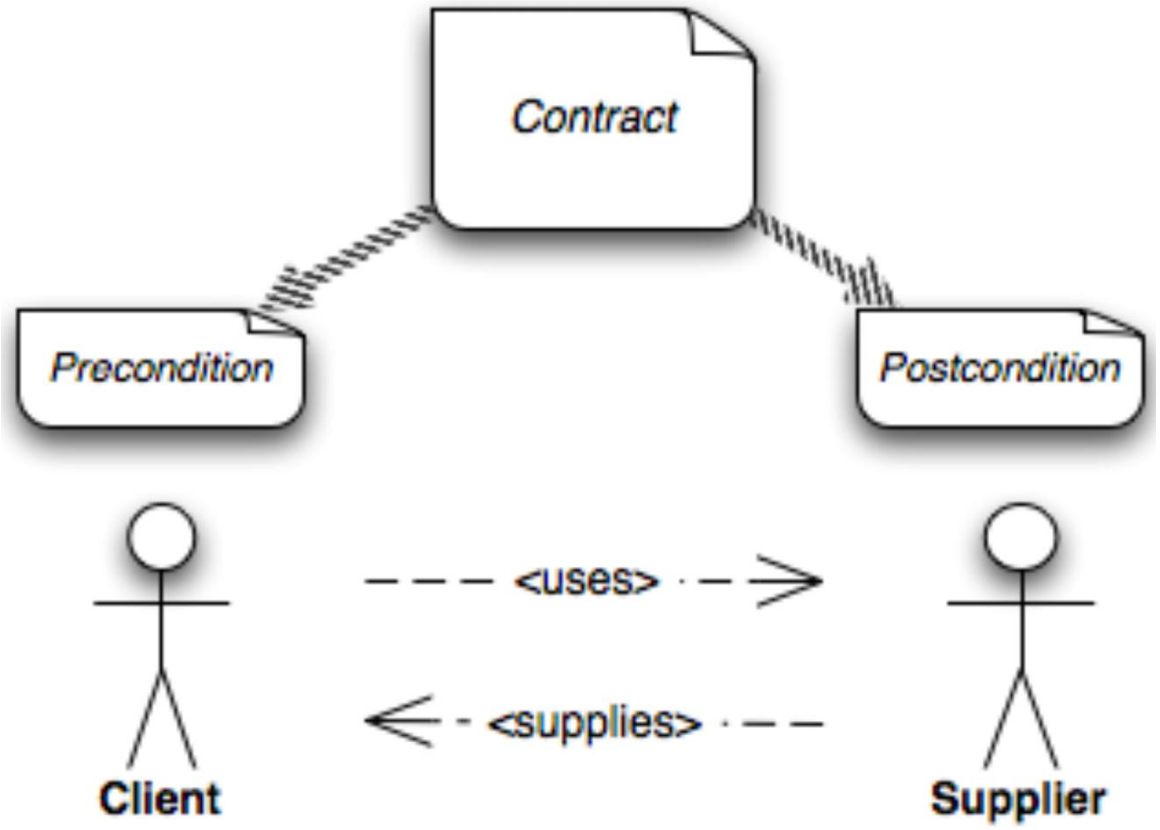
**Why would I ever use DbC?**

• Offensive programming:

– A supplier should not try to verify that the preconditions are satisfied.

• This approach differs substantially from that of defensive programming.

– A supplier verifies that preconditions are satisfied and throws an exception, if they are not.



# Offensive Programming

**Why would I ever use DbC?**

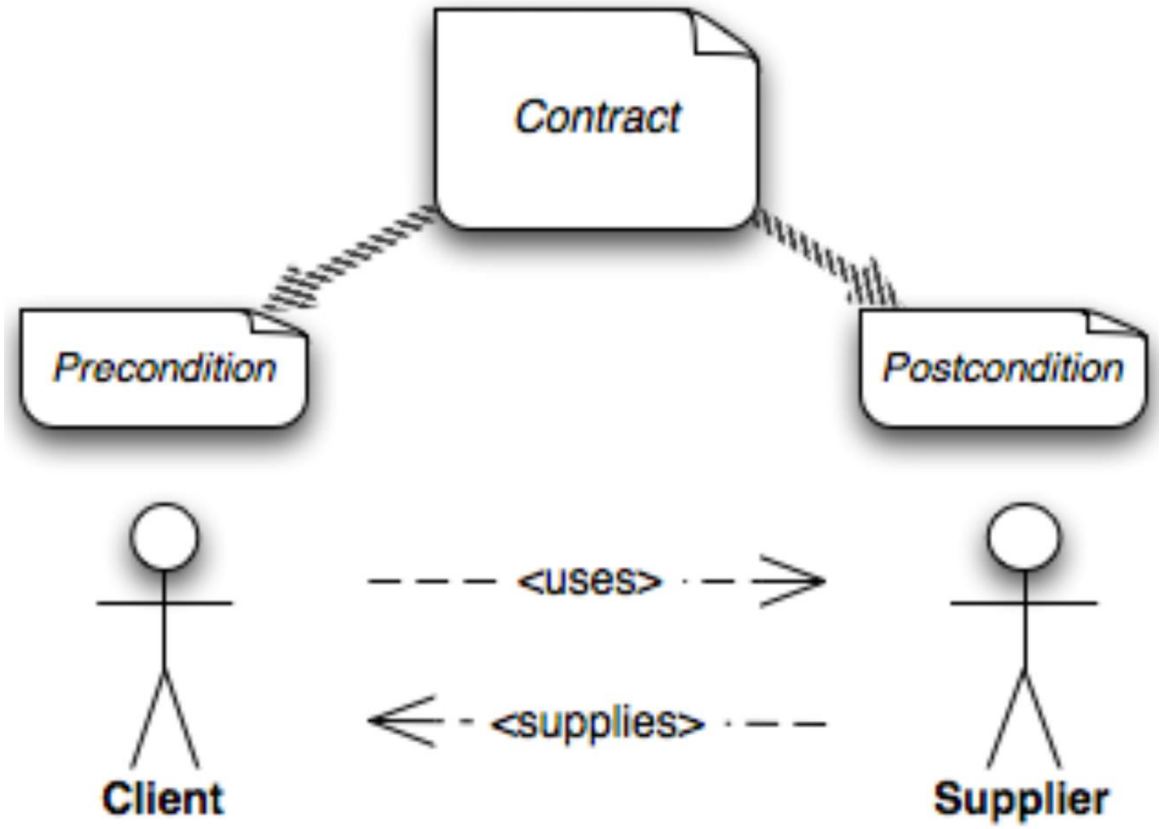
• Offensive programming makes the component run-time execution much faster.

• Examples where you might use offensive programming:

– Embedded systems

– Controller in automobile

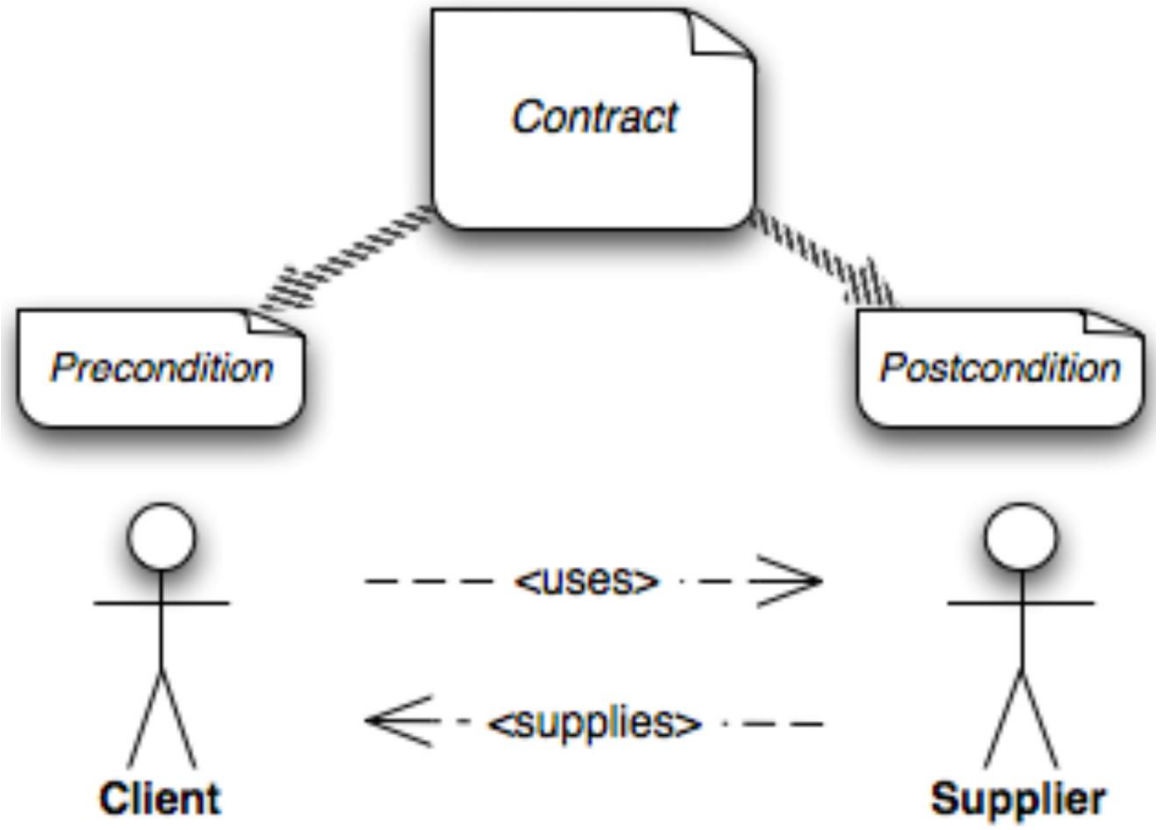
– Router



# Offensive Programming

**Performance implications:**

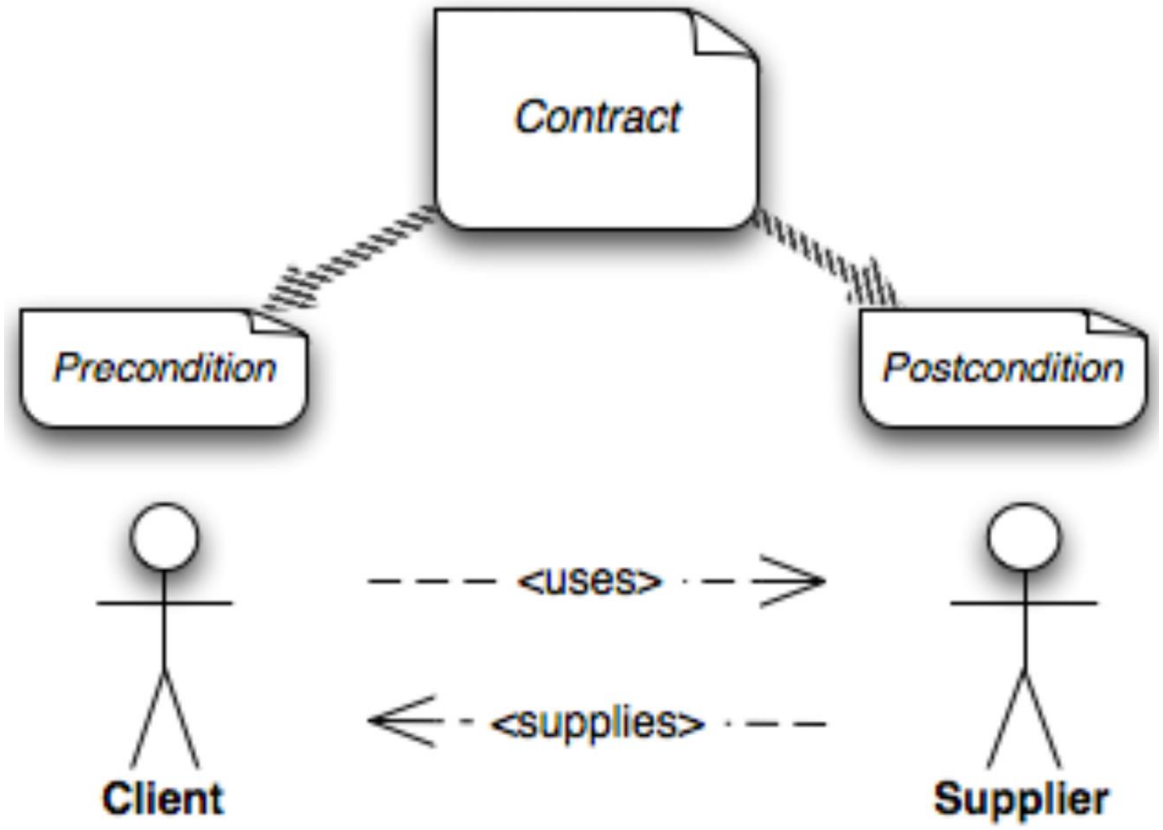
- Contract conditions should never be violated during execution of a bug-free program.
- Contracts are therefore typically only checked in debug mode during software development.
- Later at release, the contract checks are disabled to maximize performance.



# Offensive Programming

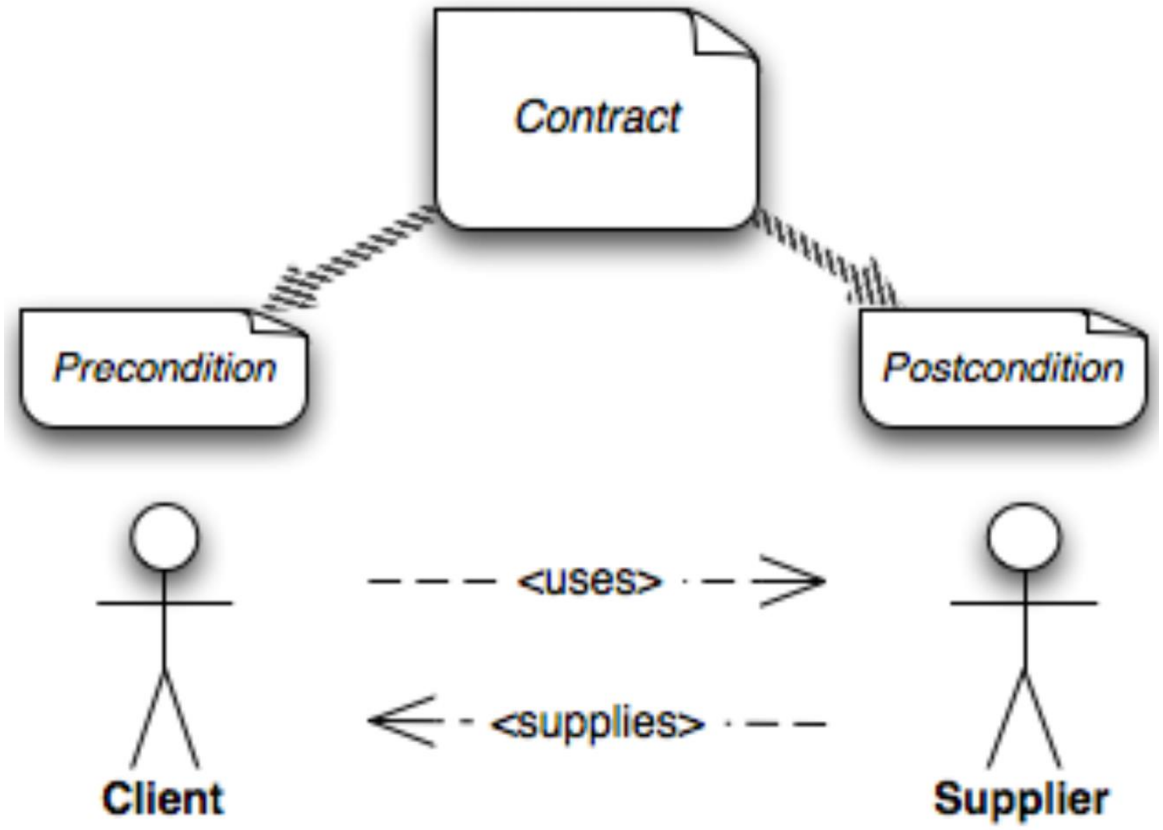
**Performance implications:**

- In many programming languages, contracts are implemented with assertions.
- Assertions are by default compiled away in release mode in C/C++, and similarly deactivated in C# and Java.
- The Python interpreter with "-O" (for "optimize") deactivates assertion checking.
- This effectively eliminates the run-time costs of assertions in production code.



# Assertions

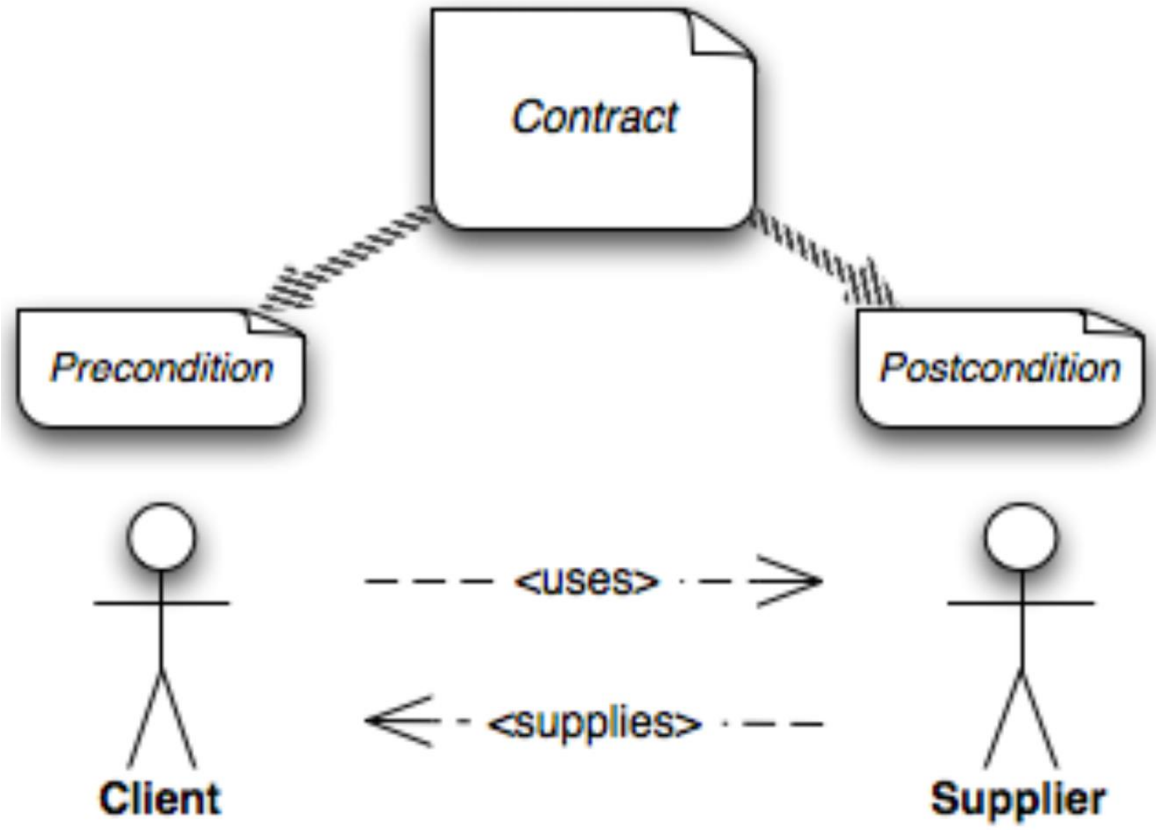
Assertions in C/C++:
• Assertions can be used to test preconditions at the beginning
• And, postconditions and invariants at the end.
• For example, we may use an assertion to check if a pointer returned by malloc() is NULL or not.
• Syntax for an assertion:
void assert( int expression );
• If expression evaluates to 0 (false), then
• the expression, source code filename, and line number are sent to the standard error, and then the abort() function is called.



# Assertions

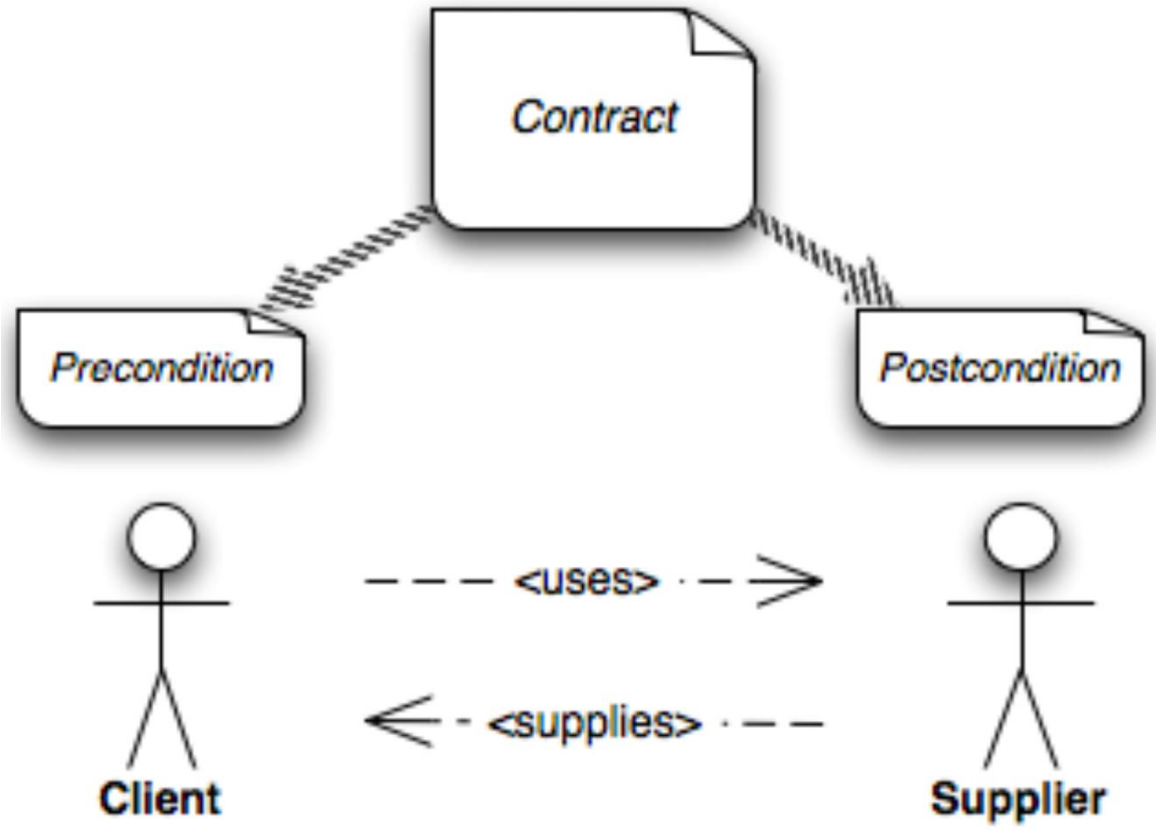
Assertions in C/C++:

```
#include <stdio.h>
#include <assert.h>
int main()
{
int x = 7;
/* Some big code in between and let's say x is accidentally changed to 9 */
x = 9;
// Programmer assumes x to be 7 in rest of the code
assert(x==7);
/* Rest of the code */
}
```



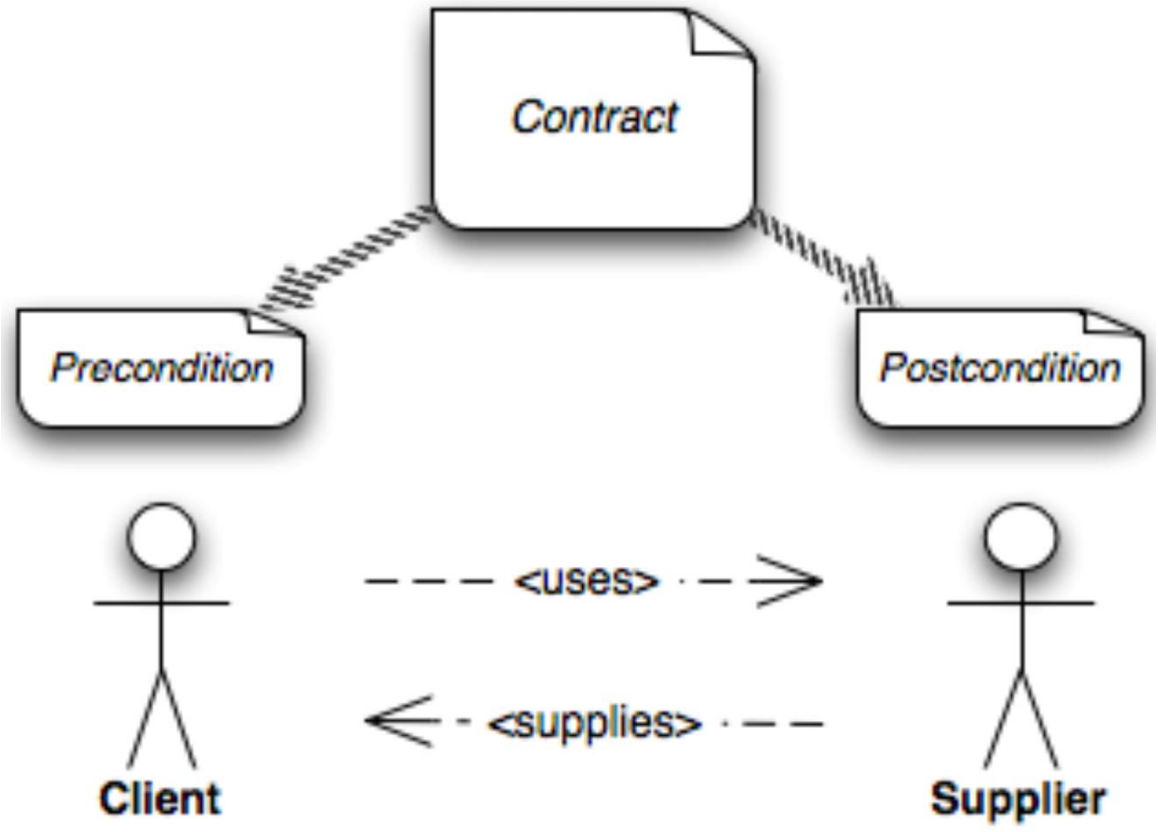
# Relationship to Software Testing

- Design by contract (DbC) does not replace regular testing strategies:
  – Unit testing
  – Integration testing
  – System testing
- Internal self-tests (via asserts) can also be activated in production code during a test-phase.
- The advantage of internal self-tests is that they can detect errors before they manifest themselves as invalid results observed by the software users.
- The use of assertions can be considered to be a way of testing the Design by Contract implementation.



# DbC and Defensive Programming

- In some environments the DbC assumption that all client components meet the preconditions is risky or not likely.
- For example:
  – User interface code
  – Library functions
- In this case, the component:
  – tests that all relevant preconditions hold true
  – replies with a suitable error message if not



# Question